



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

AIML for Robotics

Module 5

Path Planning

Section Faculty:

Raja vadhana Prabhakar

The course owner and the authors of this course deck, Prof. Raja vadhana and Dr.Sangeetha, are gratefully acknowledging all authors who made their course materials freely available online.

☐ Introduction

- ☐ Types of Robots
- ☐ Mobile Robots – Use case of Programmable solutions

☐ Components of Programmable Robotic System

- ☐ Perception
- ☐ Representation
- ☐ Motion

☐ Localization

- ☐ Taxonomy
- ☐ Types

☐ Mapping

- ☐ Feature Based
- ☐ Grid Based

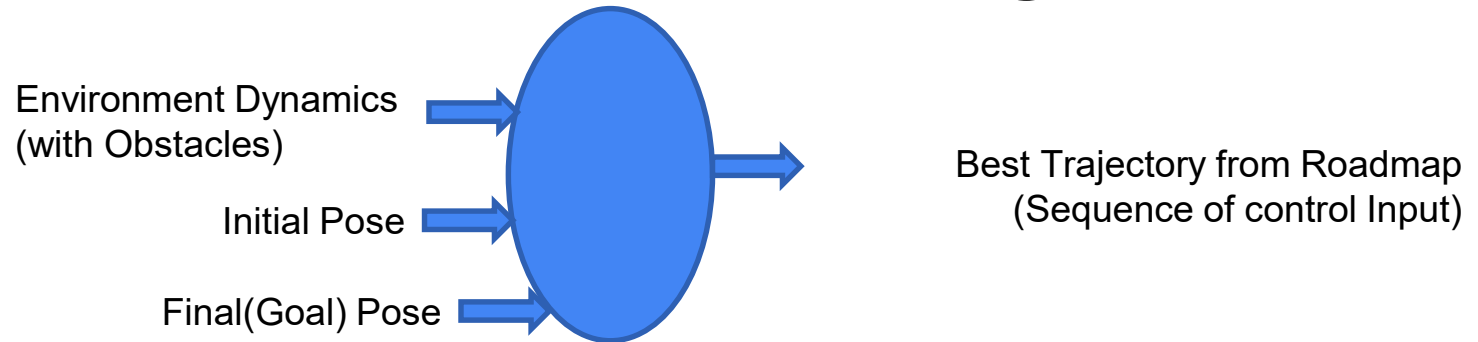
☐ Path Planning

- ☐ Graph Based
- ☐ Sample Based
- ☐ Evolutional Techniques

☐ Human – Robot Interaction

- ☐ Ethics
- ☐ Introduction to Visual Bot
- ☐ Introduction to Chat Bot

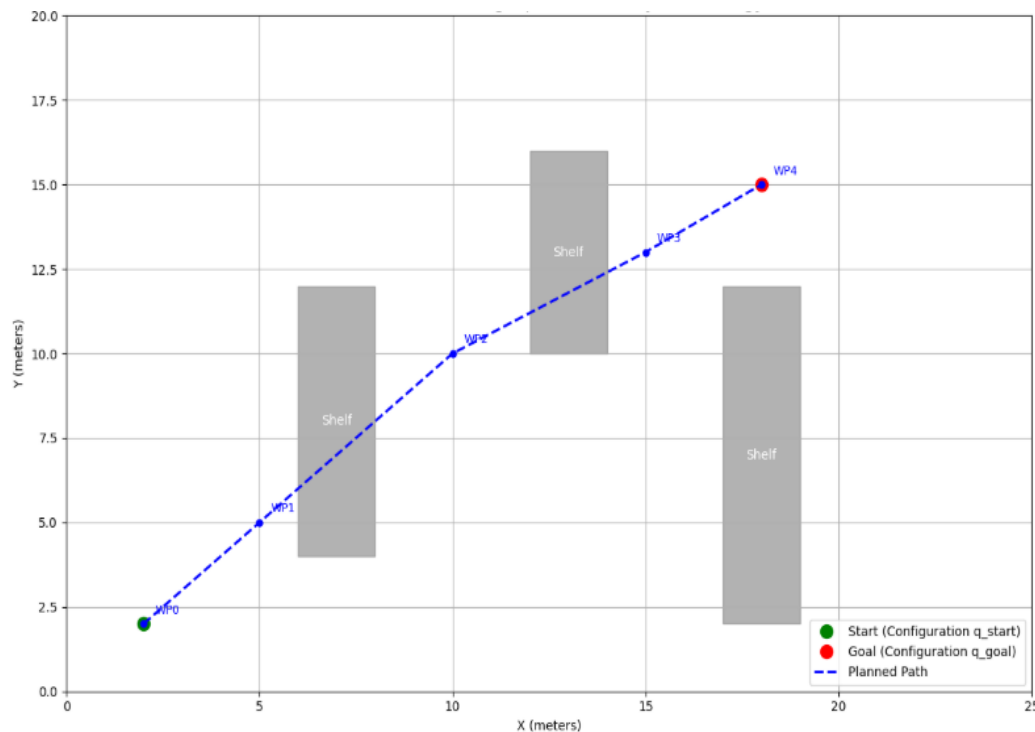
Robot Path Planning



Planning Agents



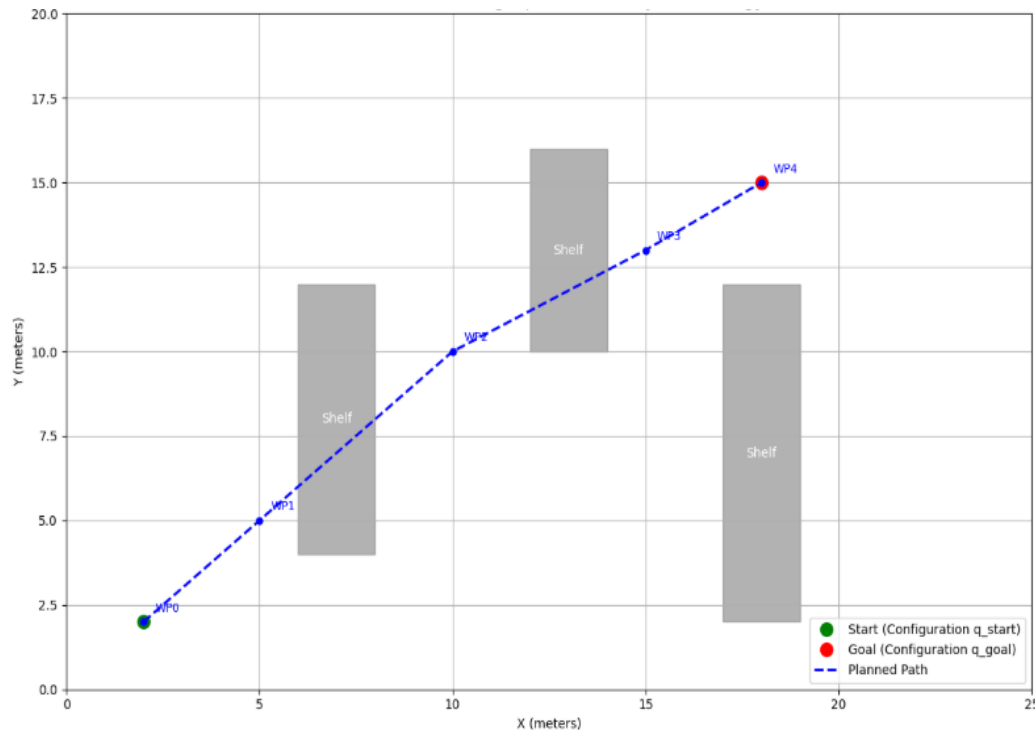
- ✓ Configuration Space
- ✓ Collision
- ✓ Map/Workspace
- ✓ Waypoints:
 $(2,2) \rightarrow (5,5) \rightarrow (10,10) \rightarrow (15,13) \rightarrow (18,15)$
- ✓ Path
- ✓ Trajectory
- ✓ Roadmap
- ✓ Sampling



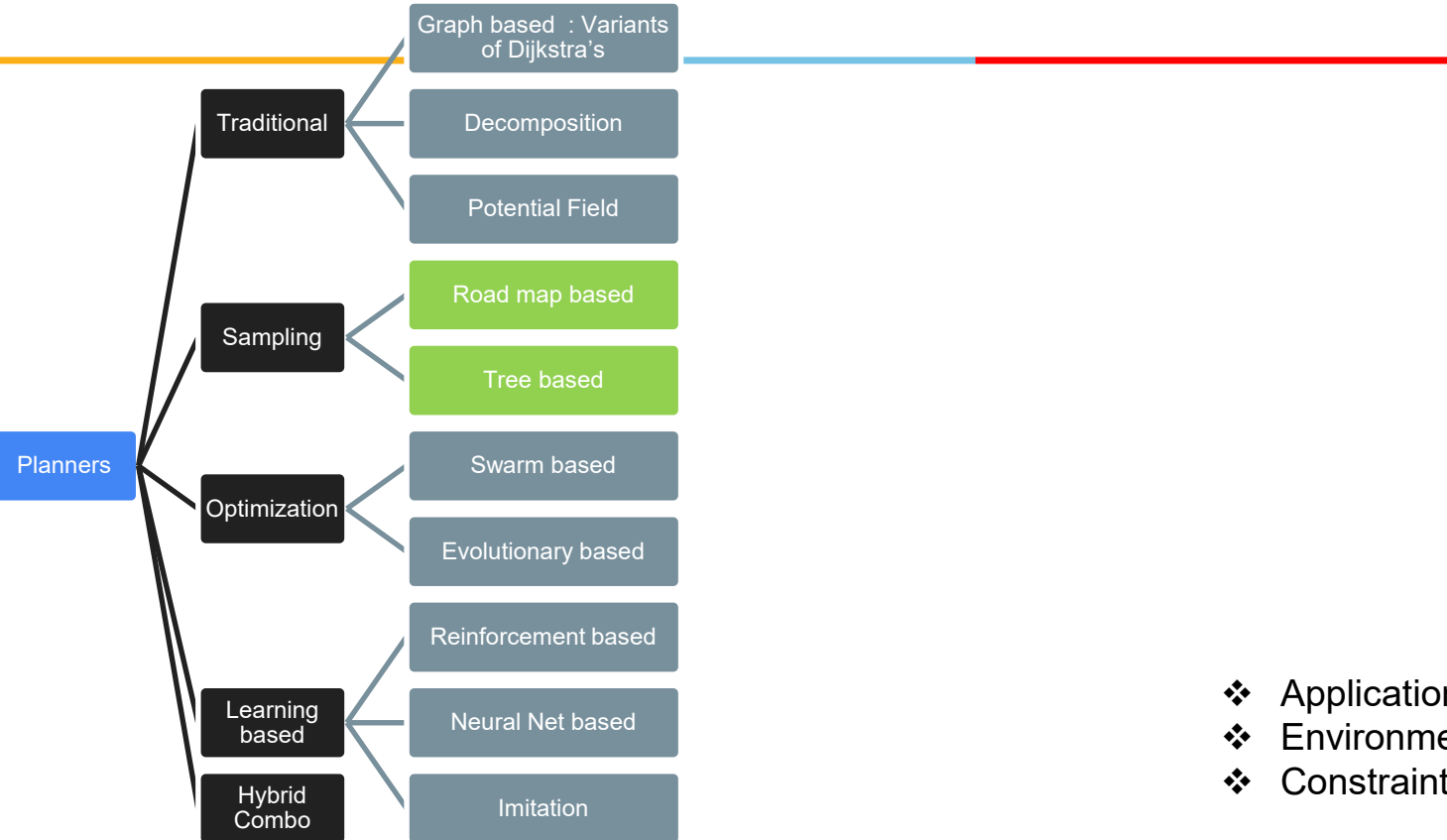
Planning Agents



- Path Length
- Cost- Heuristics
- Obstacle Proximity
- Smoothness
- Clearance
- Turn Count
- Train vs Test Accuracy



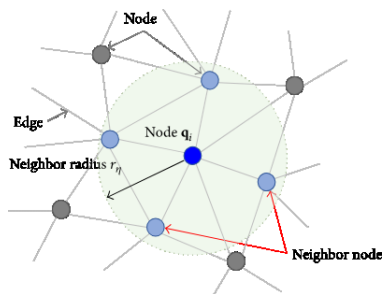
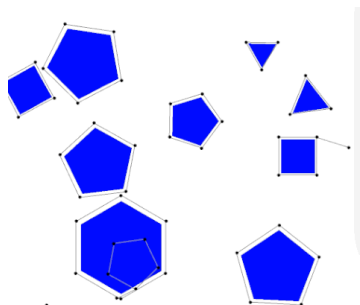
Taxonomy of Planning Problems



- ❖ Application
- ❖ Environment
- ❖ Constraints

Source Credit : <https://www.deviantart.com/>

Sampling Based Path Planning



Probabilistic Road Maps (PRM)



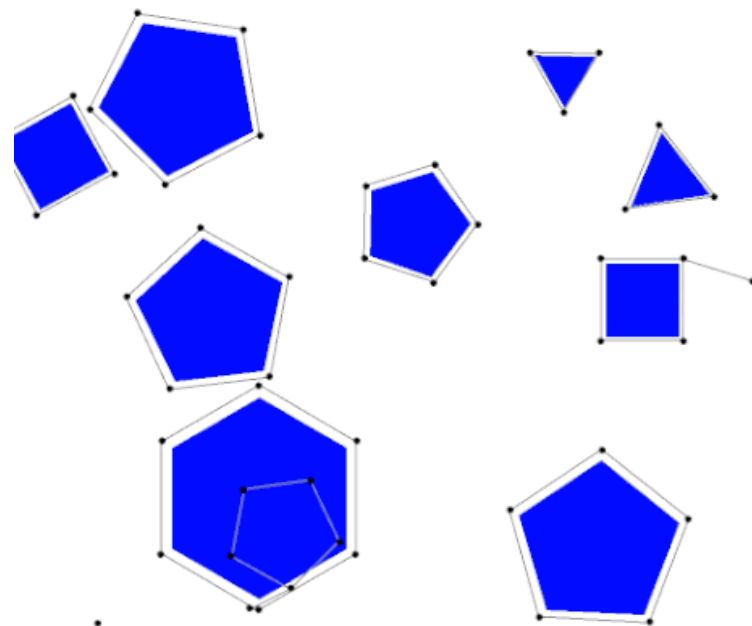
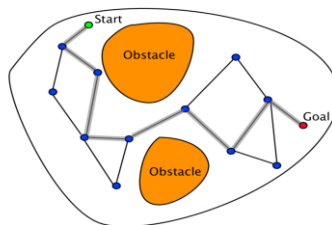
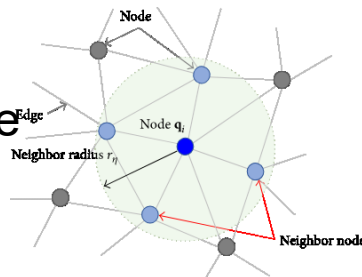
Learning (Preprocess) Phase

Random Sampling

Roadmap Construction

Forest

Querying Phase



PRM Algorithm



```
def PRM(n, r, x_init, x_goal):  
    V = []  
    E = []  
    for i in range(n):  
        x_rand = SampleFree()  
        U = Near(V, E, x_rand, r)  
        V.append(x_rand)  
        U.Sort(distance(x_rand))  
        for u in U:  
            if [[x_rand, u], [u, x_rand]] not in E:  
                if CollisionFree(x_rand, u):  
                    E.append([[x_rand, u], [u, x_rand]])  
  
    return V, E
```

Batch PRM* Algorithm



```
def PRM(n, gamma, x_init, x_goal):
    V = []
    E = []
    for i in range(n):
        x_rand = SampleFree()
        r = gamma * (log(n)/n)^(1/d)
        # d= dimension , gamma = Percolation threshold = ~1.5 or 2
        U = Near(V, E, x_rand, r)
        V.append(x_rand)
        U.Sort(distance(x_rand))
        for u in U:
            if [[x_rand, u], [u, x_rand]] not in E:
                if CollisionFree(x_rand, u):
                    E.append([[x_rand, u], [u, x_rand]])

    return V, E
```

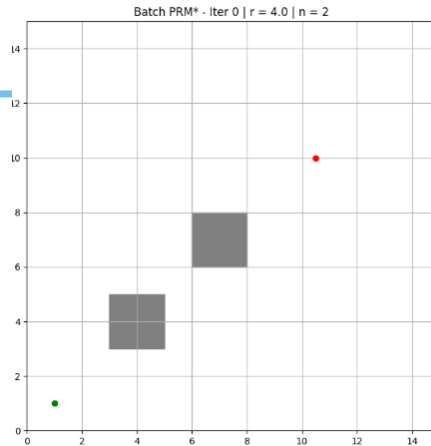
Incremental PRM* Algorithm



```
def PRM(gamma, x_init, x_goal):
    V = []
    E = []
    n = 0
    While not resource limit exceeded :
        x_rand = SampleFree()
        n = n+1
        r = gamma * (log(n)/n)^(1/d)
        # d= dimension , gamma = Percolation threshold = ~1.5 or 2
        U = Near(V, E, x_rand, r)
        V.append(x_rand)
        U.Sort(distance(x_rand))
        for u in U:
            if [[x_rand, u], [u, x_rand]] not in E:
                if CollisionFree(x_rand, u):
                    E.append([[x_rand, u], [u, x_rand]])

    return V, E
```

1. Initialize
2. Generate Sample Particles
3. Search Nearest Neighbor
4. Check Collision Free Path
5. Construct the Graph
6. Query via Graph Search
7. Goal Test
8. Repeat



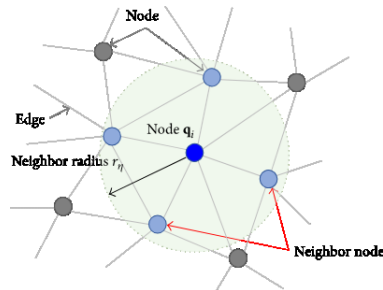
Batch PRM* Algorithm

Algorithm 4: PRM*

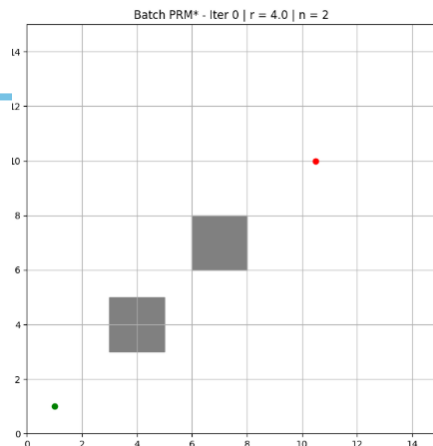
```

1  $V \leftarrow \{x_{init}\} \cup \{\text{SampleFree}_i\}_{i=1,\dots,n}; E \leftarrow \emptyset;$ 
2 foreach  $v \in V$  do
3    $U \leftarrow \text{Near}(G = (V, E), v, \gamma_{\text{PRM}}(\log(n)/n)^{1/d} \setminus \{v\});$ 
4   foreach  $u \in U$  do
5     if  $\text{CollisionFree}(v, u)$  then  $E \leftarrow E \cup \{(v, u), (u, v)\}$ 
6 return  $G = (V, E);$ 

```



1. Initialize



Batch PRM* Algorithm

Algorithm 4: PRM*

```

1  $V \leftarrow \{x_{\text{init}}\} \cup \{\text{SampleFree}_i\}_{i=1,\dots,n}; E \leftarrow \emptyset;$ 
2 foreach  $v \in V$  do
3    $U \leftarrow \text{Near}(G = (V, E), v, \gamma_{\text{PRM}}(\log(n)/n)^{1/d} \setminus \{v\});$ 
4   foreach  $u \in U$  do
5     if  $\text{CollisionFree}(v, u)$  then  $E \leftarrow E \cup \{(v, u), (u, v)\}$ 
6 return  $G = (V, E);$ 

```

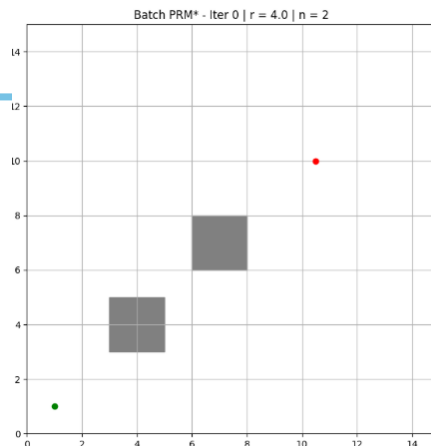
$d = 2$

$\Gamma = 8$

Start = (1,1)

Goal = (10.5, 10)

1. Initialize
2. Generate Sample Particles



Batch PRM* Algorithm

Algorithm 4: PRM*

```

1  $V \leftarrow \{x_{init}\} \cup \{\text{SampleFree}_i\}_{i=1,\dots,n}; E \leftarrow \emptyset;$ 
2 foreach  $v \in V$  do
3    $U \leftarrow \text{Near}(G = (V, E), v, \gamma_{\text{PRM}}(\log(n)/n)^{1/d}) \setminus \{v\};$ 
4   foreach  $u \in U$  do
5     if  $\text{CollisionFree}(v, u)$  then  $E \leftarrow E \cup \{(v, u), (u, v)\}$ 
6 return  $G = (V, E);$ 

```

$d = 2$

$\Gamma = 8$

Start = (1, 1)

Goal = (10.5, 10)

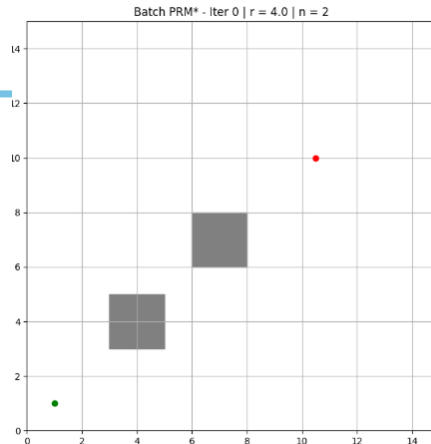
$N = 2$

$R = \sim 4.7$

5 Samples X_r :

$\{(2, 2), S2 = (4, 4), S3 = (7, 4), S4 = (9, 9), S5 = (11, 10)\}$

1. Initialize
2. Generate Sample Particles
3. Search Nearest Neighbor
4. Check Collision Free Path



Batch PRM* Algorithm

Algorithm 4: PRM*

```

1  $V \leftarrow \{x_{init}\} \cup \{\text{SampleFree}_i\}_{i=1,\dots,n}; E \leftarrow \emptyset;$ 
2 foreach  $v \in V$  do
3    $U \leftarrow \text{Near}(G = (V, E), v, \gamma_{\text{PRM}}(\log(n)/n)^{1/d}) \setminus \{v\};$ 
4   foreach  $u \in U$  do
5     if  $\text{CollisionFree}(v, u)$  then  $E \leftarrow E \cup \{(v, u), (u, v)\}$ 
6 return  $G = (V, E);$ 

```

$d = 2$

$\Gamma = 8$

Start = (1,1)

Goal = (10.5, 10)

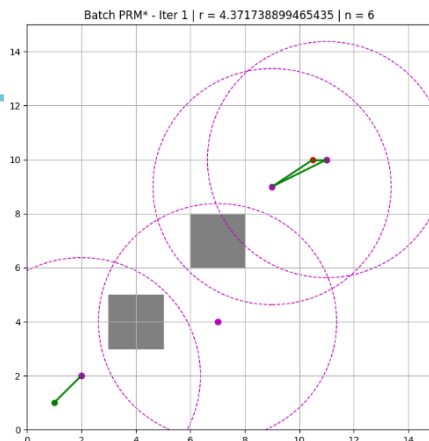
$N = 6$

$R = \sim 4.7$

5 Samples X_r :

$\{S1 = (2, 2), S2 = (4, 4), S3 = (7, 4), S4 = (9, 9), S5 = (11, 10)\}$

1. Initialize
2. Generate Sample Particles
3. Search Nearest Neighbor
4. Check Collision Free Path
5. Construct the Graph



Batch PRM* Algorithm

Algorithm 4: PRM*

```

1  $V \leftarrow \{x_{init}\} \cup \{\text{SampleFree}_i\}_{i=1,\dots,n}; E \leftarrow \emptyset;$ 
2 foreach  $v \in V$  do
3    $U \leftarrow \text{Near}(G = (V, E), v, \gamma_{\text{PRM}}(\log(n)/n)^{1/d} \setminus \{v\});$ 
4   foreach  $u \in U$  do
5     if  $\text{CollisionFree}(v, u)$  then  $E \leftarrow E \cup \{(v, u), (u, v)\}$ 
6 return  $G = (V, E);$ 

```

$d = 2$

$\Gamma = 8$

Start = (1,1)

Goal = (10.5, 10)

$N = 6$

$R = \sim 4.37$

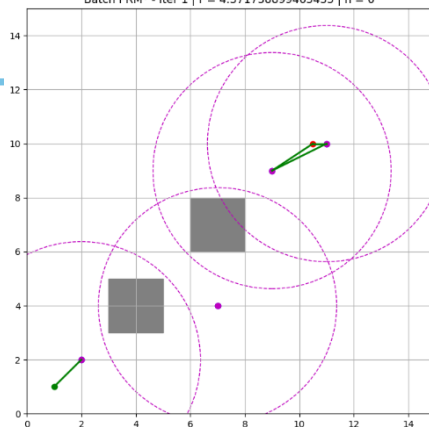
5 Samples X_r :

$\{S1 = (2, 2), S2 = (4, 4), S3 = (7, 4), S4 = (9, 9), S5 = (11, 10)\}$

Eg., $S3 \leftrightarrow S4$, $S0 \leftrightarrow S3$: rejected

Note: Collision logic can be fine tuned. Assume point position free for simplicity

Batch PRM* - iter 1 | $r = 4.371738899465435$ | $n = 6$



Batch PRM* Algorithm

Algorithm 4: PRM*

```

1  $V \leftarrow \{x_{init}\} \cup \{\text{SampleFree}_i\}_{i=1,\dots,n}; E \leftarrow \emptyset;$ 
2 foreach  $v \in V$  do
3    $U \leftarrow \text{Near}(G = (V, E), v, \gamma_{\text{PRM}}(\log(n)/n)^{1/d} \setminus \{v\});$ 
4   foreach  $u \in U$  do
5     if  $\text{CollisionFree}(v, u)$  then  $E \leftarrow E \cup \{(v, u), (u, v)\}$ 
6 return  $G = (V, E);$ 

```

$d = 2$

$\Gamma = 8$

Start = (1,1)

$N = 6$

Goal = (10.5, 10)

$R = \sim 4.37$

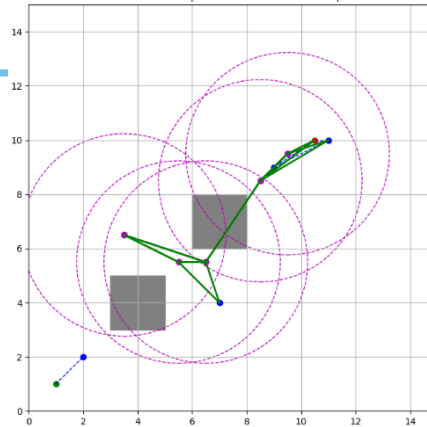
$\{S1=(2, 2), S2=(4, 4), S3=(7, 4), S4=(9, 9), S5=(11, 10)\}$

5 Samples X_r :

$\{S2=(5.5, 5.5), S6=(6.5, 5.5), S7=(8.5, 8.5), S8=(9.5, 9.5), S9=(3.5, 6.5)\}$

1. Initialize
2. **Generate Sample Particles**
3. Search Nearest Neighbor
4. Check Collision Free Path
5. Construct the Graph

Batch PRM* - Iter 2 | $r = 3.735156044678671$ | $n = 11$



1. Initialize
2. Generate Sample Particles
3. Search Nearest Neighbor
4. Check Collision Free Path
5. Construct the Graph

Batch PRM* Algorithm

Algorithm 4: PRM*

```

1  $V \leftarrow \{x_{init}\} \cup \{\text{SampleFree}_i\}_{i=1,\dots,n}; E \leftarrow \emptyset;$ 
2 foreach  $v \in V$  do
3    $U \leftarrow \text{Near}(G = (V, E), v, \gamma_{\text{PRM}}(\log(n)/n)^{1/d} \setminus \{v\});$ 
4   foreach  $u \in U$  do
5     if  $\text{CollisionFree}(v, u)$  then  $E \leftarrow E \cup \{(v, u), (u, v)\}$ 
6 return  $G = (V, E);$ 

```

$d = 2$

$\Gamma = 8$

Start = (1,1)

$N = 11$

Goal = (10.5, 10)

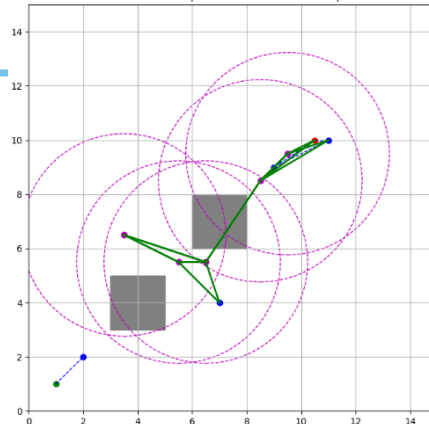
$R = \sim 3.74$

$\{S1=(2, 2), S2=(4, 4), S3=(7, 4), S4=(9, 9), S5=(11, 10)\}$

5 Samples X_r :

$\{S2=(5.5, 5.5), S6=(6.5, 5.5), S7=(8.5, 8.5), S8=(9.5, 9.5), S9=(3.5, 6.5)\}$

Batch PRM* - Iter 2 | $r = 3.735156044678671$ | $n = 11$



Batch PRM* Algorithm

Algorithm 4: PRM*

```

1  $V \leftarrow \{x_{\text{init}}\} \cup \{\text{SampleFree}_i\}_{i=1,\dots,n}; E \leftarrow \emptyset;$ 
2 foreach  $v \in V$  do
3    $U \leftarrow \text{Near}(G = (V, E), v, \gamma_{\text{PRM}}(\log(n)/n)^{1/d}) \setminus \{v\};$ 
4   foreach  $u \in U$  do
5     if  $\text{CollisionFree}(v, u)$  then  $E \leftarrow E \cup \{(v, u), (u, v)\}$ 
6 return  $G = (V, E);$ 

```

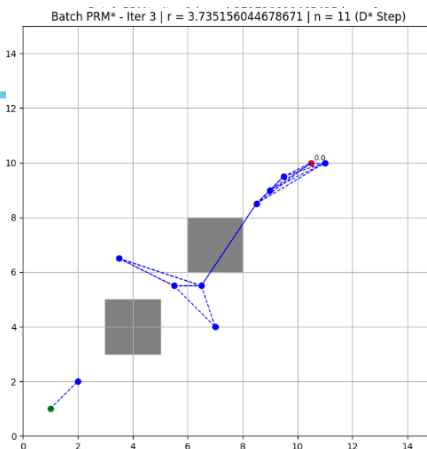
Start = (1,1)

Goal = (10.5, 10)

$K=f=g+h$
 $g(\text{Goal})=0$

1. Initialize
2. Generate Sample Particles
3. Search Nearest Neighbor
4. Check Collision Free Path
5. Construct the Graph
6. Query via Graph Search

1. Initialize
2. Generate Sample Particles
3. Search Nearest Neighbor
4. Check Collision Free Path
5. Construct the Graph
6. Query via Graph Search



Batch PRM* Algorithm

Algorithm 4: PRM*

```

1  $V \leftarrow \{x_{init}\} \cup \{\text{SampleFree}_i\}_{i=1,\dots,n}; E \leftarrow \emptyset;$ 
2 foreach  $v \in V$  do
3    $U \leftarrow \text{Near}(G = (V, E), v, \gamma_{\text{PRM}}(\log(n)/n)^{1/d}) \setminus \{v\};$ 
4   foreach  $u \in U$  do
5     if  $\text{CollisionFree}(v, u)$  then  $E \leftarrow E \cup \{(v, u), (u, v)\}$ 
6 return  $G = (V, E);$ 

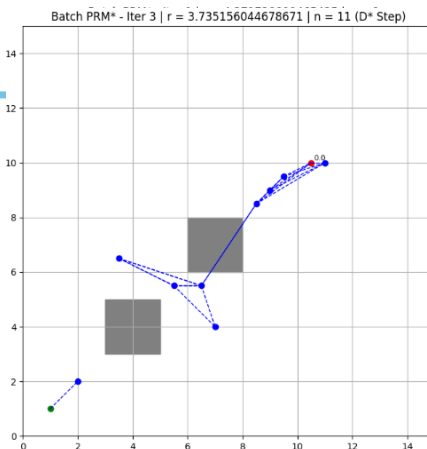
```

Start = (1,1)

Goal = (10.5, 10)

$K=f=g+h$
 $g(\text{Goal})=0$

1. Initialize
2. Generate Sample Particles
3. Search Nearest Neighbor
4. Check Collision Free Path
5. Construct the Graph
6. Query via Graph Search



Batch PRM* Algorithm

Algorithm 4: PRM*

```

1  $V \leftarrow \{x_{init}\} \cup \{\text{SampleFree}_i\}_{i=1,\dots,n}; E \leftarrow \emptyset;$ 
2 foreach  $v \in V$  do
3    $U \leftarrow \text{Near}(G = (V, E), v, \gamma_{\text{PRM}}(\log(n)/n)^{1/d}) \setminus \{v\};$ 
4   foreach  $u \in U$  do
5     if  $\text{CollisionFree}(v, u)$  then  $E \leftarrow E \cup \{(v, u), (u, v)\}$ 
6 return  $G = (V, E);$ 

```

Start = (1,1)
Goal = (10.5, 10)

K=f+g+h
g(Goal)=0

Priority Queue (minHeap)	
Node (11, 10)	k = 0.50
Node (9.5, 9.5)	k = 1.12
Node (5.5, 5.5)	k = 6.73
Node (8.5, 8.5)	k = 2.50
Node (9, 9)	k = 1.80
Node (2, 2)	k = 11.67
Node (3.5, 6.5)	k = 7.83
Node (6.5, 5.5)	k = 6.02
Node (7, 4)	k = 6.95
Node (1, 1)	k = 13.09

Batch PRM* Algorithm

Algorithm 4: PRM*

```

1  $V \leftarrow \{x_{init}\} \cup \{\text{SampleFree}_i\}_{i=1,\dots,n}; E \leftarrow \emptyset;$ 
2 foreach  $v \in V$  do
3    $U \leftarrow \text{Near}(G = (V, E), v, \gamma_{\text{PRM}}(\log(n)/n)^{1/d} \setminus \{v\});$ 
4   foreach  $u \in U$  do
5     if  $\text{CollisionFree}(v, u)$  then  $E \leftarrow E \cup \{(v, u), (u, v)\}$ 
6 return  $G = (V, E);$ 

```

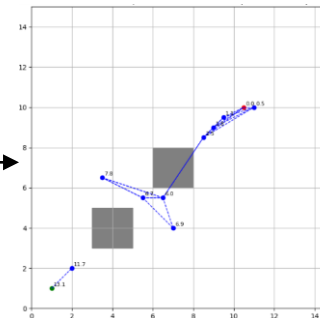
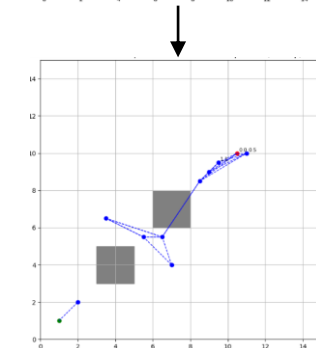
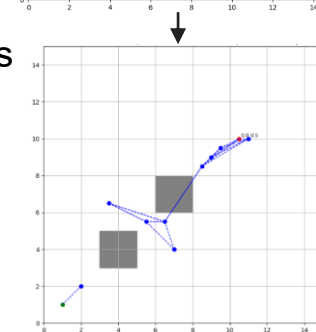
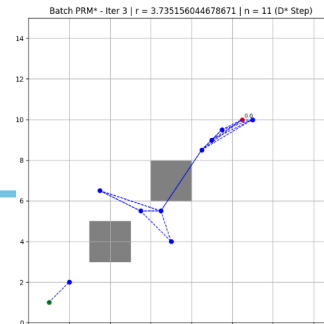
Start = (1,1)

Goal = (10.5, 10)

$K=f+g+h$

$g(\text{Goal})=0$

1. Initialize
2. Generate Sample Particles
3. Search Nearest Neighbor
4. Check Collision Free Path
5. Construct the Graph
6. Query via Graph Search



Priority Queue (minHeap)	
Node (11, 10)	k = 0.50
Node (9.5, 9.5)	k = 1.12
Node (5.5, 5.5)	k = 6.73
Node (8.5, 8.5)	k = 2.50
Node (9, 9)	k = 1.80
Node (2, 2)	k = 11.67
Node (3.5, 6.5)	k = 7.83
Node (6.5, 5.5)	k = 6.02
Node (7, 4)	k = 6.95
Node (1, 1)	k = 13.09

Batch PRM* Algorithm

Algorithm 4: PRM*

```

1  $V \leftarrow \{x_{init}\} \cup \{\text{SampleFree}_i\}_{i=1,\dots,n}; E \leftarrow \emptyset;$ 
2 foreach  $v \in V$  do
3    $U \leftarrow \text{Near}(G = (V, E), v, \gamma_{\text{PRM}}(\log(n)/n)^{1/d} \setminus \{v\});$ 
4   foreach  $u \in U$  do
5     if  $\text{CollisionFree}(v, u)$  then  $E \leftarrow E \cup \{(v, u), (u, v)\}$ 
6 return  $G = (V, E);$ 

```

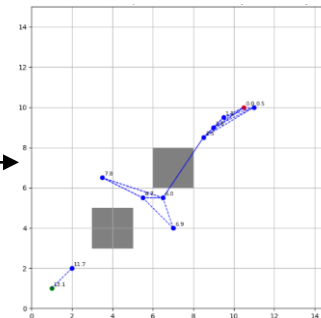
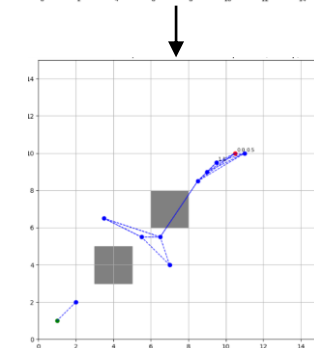
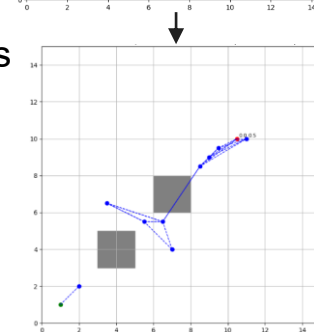
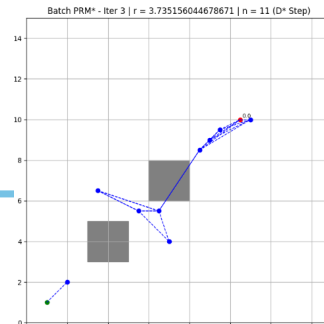
Start = (1,1)

Goal = (10.5, 10)

$K=f+g+h$

$g(\text{Goal})=0$

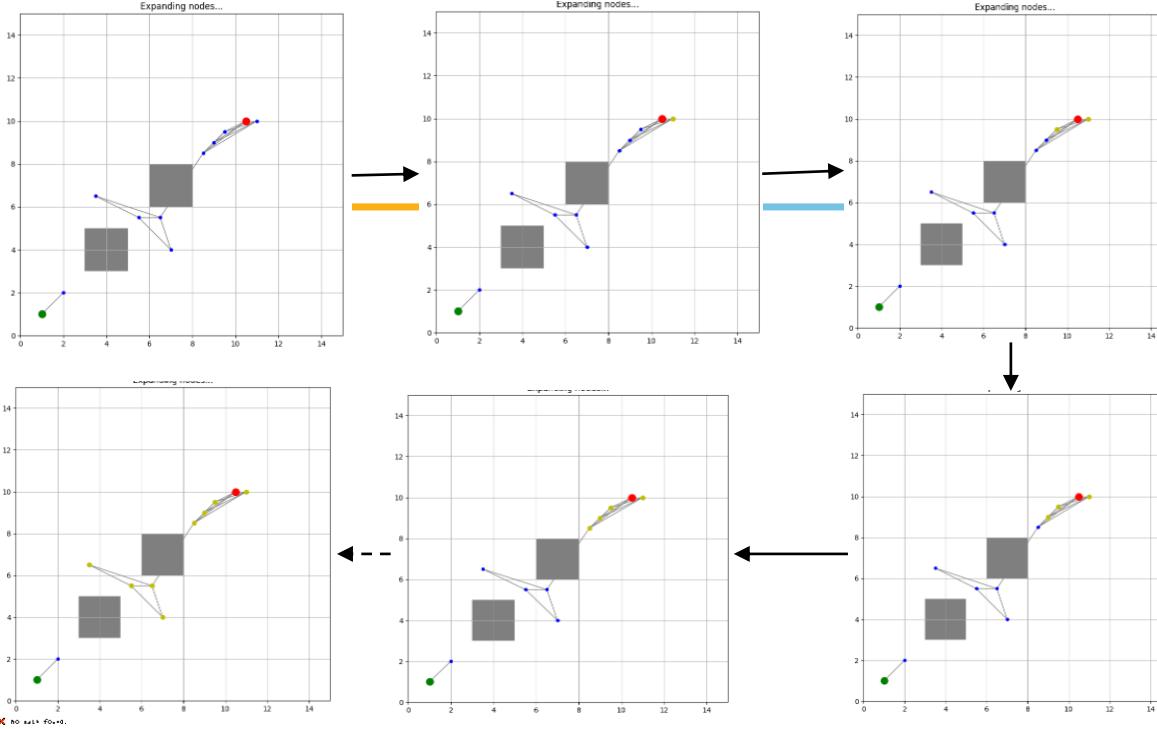
1. Initialize
2. Generate Sample Particles
3. Search Nearest Neighbor
4. Check Collision Free Path
5. Construct the Graph
6. Query via Graph Search
7. **Goal Test**
8. Repeat



Priority Queue (minHeap)	
Node (8.5, 8.5)	k = 13.11
Node (9, 9)	k = 13.12
Node (6.5, 5.5)	k = 13.21
Node (11, 10)	k = 13.95
Node (9.5, 9.5)	k = 13.14
Node (7, 4)	k = 14.39
Node (5.5, 5.5)	k = 13.47
Node (3.5, 6.5)	k = 15.31

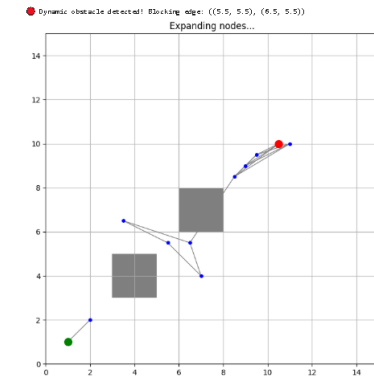


D* Algorithm



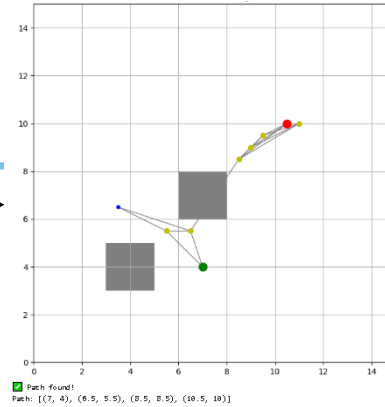
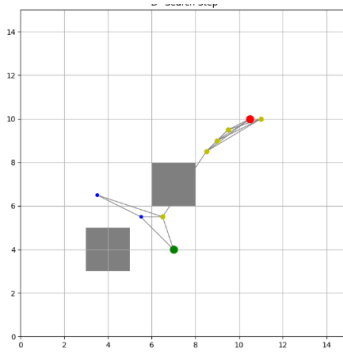
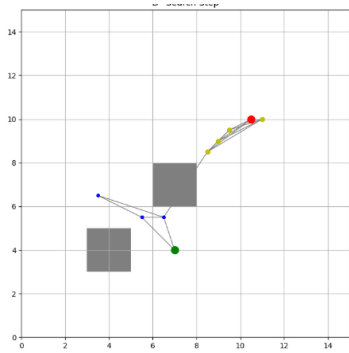
Start = (1,1)
Goal = (10.5, 10)

$K=f+g$
 $g(\text{Goal})=0$



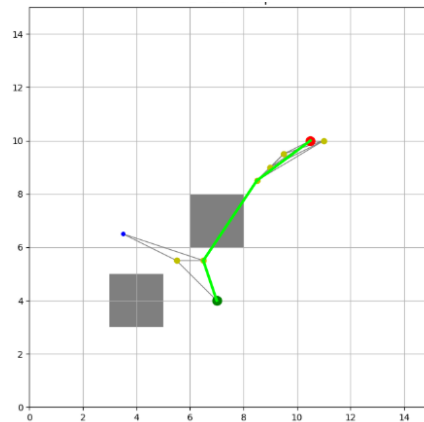


D* Algorithm



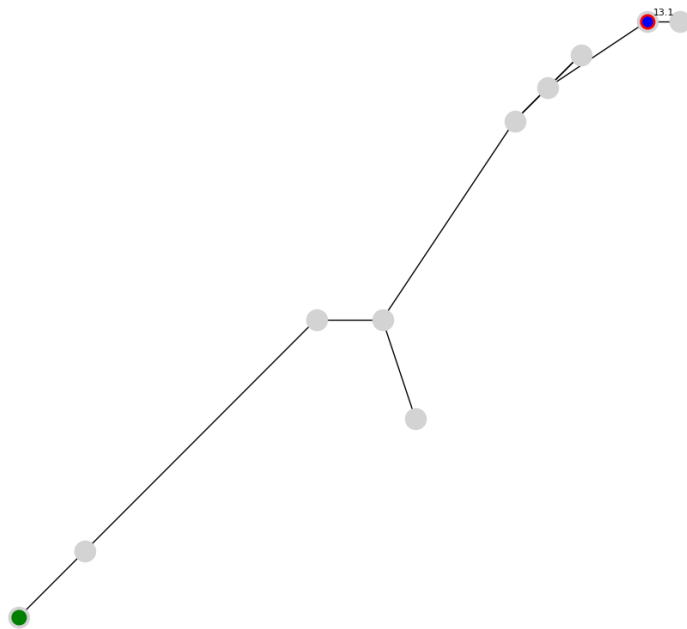
Start = (7,4)
Goal = (10.5, 10)

$K=f=g+h$
 $g(\text{Goal})=0$



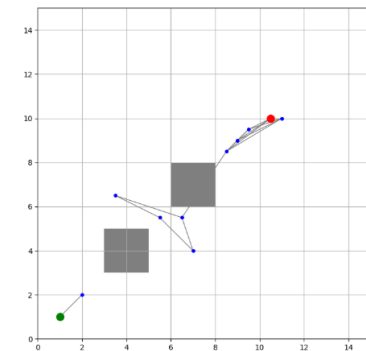
Note: The start node is modified here for different run of the algorithm

D* Algorithm



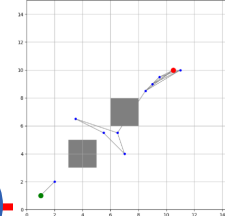
Start = (7,4)
Goal = (10.5, 10)

$K=f+g+h$
 $g(\text{Goal})=0$



Expanding (10.5, 10) (k=13.1)

Start
Goal



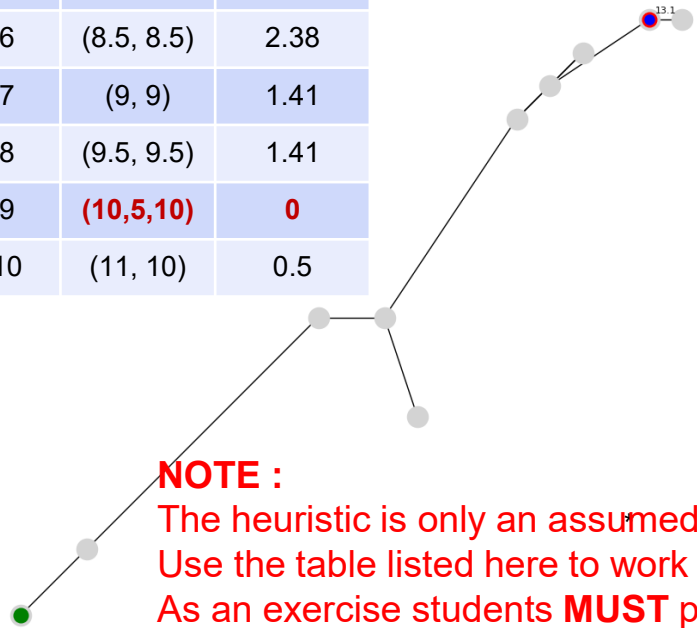
D* Algorithm

Start = (1,1)
Goal = (10.5, 10)

$K=f=g+h$
 $g(\text{Goal})=0$

9

n	State	h(n)
1	(1,1)	13.45
2	(2,2)	11.76
3	(5.5, 5.5)	7.78
4	(6.5, 6.5)	6.73
5	(7,4)	6.86
6	(8.5, 8.5)	2.38
7	(9, 9)	1.41
8	(9.5, 9.5)	1.41
9	(10.5,10)	0
10	(11, 10)	0.5



NOTE :

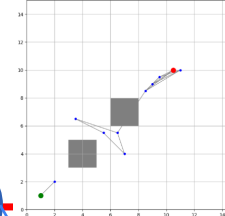
The heuristic is only an assumed dummy value suitable only for A*.

Use the table listed here to work out the A* algorithm.

As an exercise students **MUST** practice with $h(n) = \text{distance}(n, \text{Start})$ and redo the D* problem. This was discussed in detail during the live session.

Expanding (10.5, 10) (k=13.1)

Start
Goal



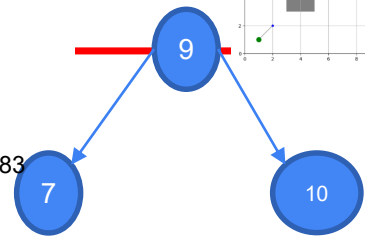
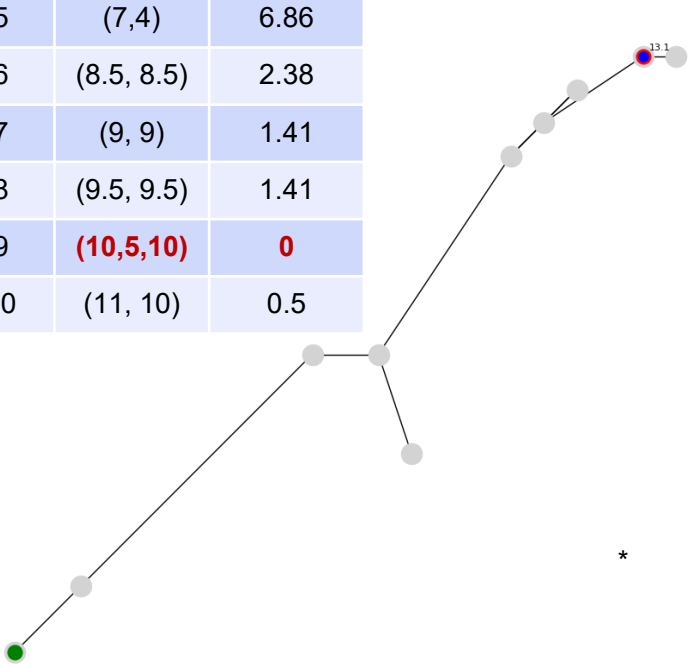
D* Algorithm

Start = (1,1)
Goal = (10.5, 10)
 $K=f=g+h$
 $g(\text{Goal})=0$

$1.41+1.41 = 2.83$

$0.5+0.5=1$

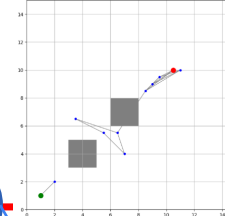
n	State	h(n)
1	(1,1)	13.45
2	(2,2)	11.76
3	(5.5, 5.5)	7.78
4	(6.5, 6.5)	6.73
5	(7,4)	6.86
6	(8.5, 8.5)	2.38
7	(9, 9)	1.41
8	(9.5, 9.5)	1.41
9	(10.5,10)	0
10	(11, 10)	0.5



*

Expanding (10.5, 10) (k=13.1)

Start
Goal

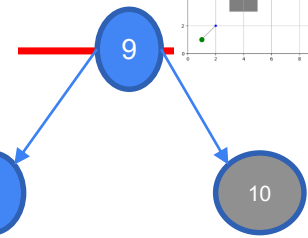


D* Algorithm

Start = (1,1)
Goal = (10.5, 10)
 $K=f=g+h$
 $g(\text{Goal})=0$

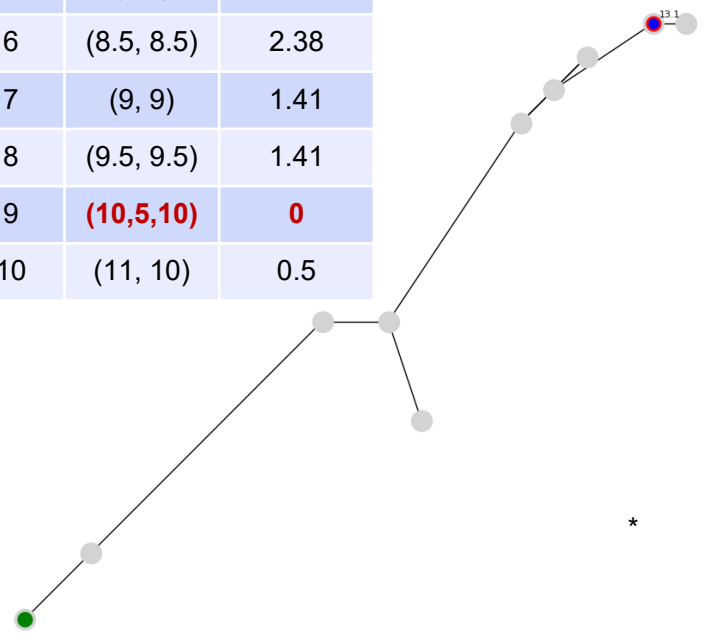
$1.41+1.41 = 2.83$

$0.5+0.5=1$



*

n	State	h(n)
1	(1,1)	13.45
2	(2,2)	11.76
3	(5.5, 5.5)	7.78
4	(6.5, 6.5)	6.73
5	(7,4)	6.86
6	(8.5, 8.5)	2.38
7	(9, 9)	1.41
8	(9.5, 9.5)	1.41
9	(10.5,10)	0
10	(11, 10)	0.5



Expanding (10.5, 10) (k=13.1)

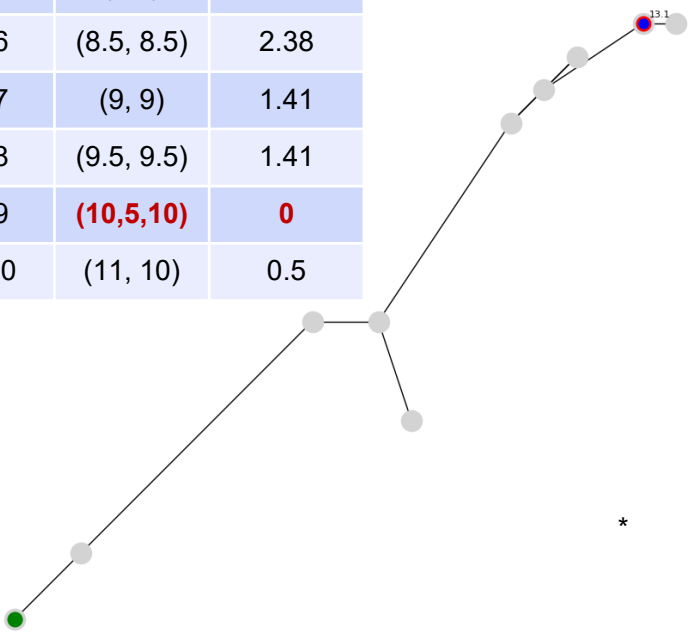
Start
Goal



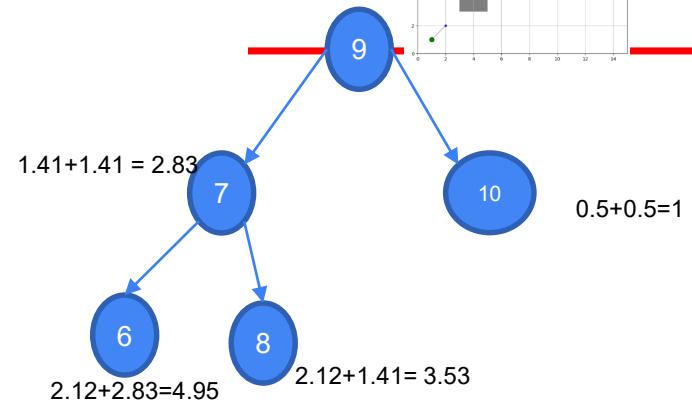
D* Algorithm

Start = (1,1)
Goal = (10.5, 10)
 $K=f=g+h$
 $g(\text{Goal})=0$

n	State	h(n)
1	(1,1)	13.45
2	(2,2)	11.76
3	(5.5, 5.5)	7.78
4	(6.5, 6.5)	6.73
5	(7,4)	6.86
6	(8.5, 8.5)	2.38
7	(9, 9)	1.41
8	(9.5, 9.5)	1.41
9	(10.5,10)	0
10	(11, 10)	0.5



*



Expanding (10.5, 10) (k=13.1)

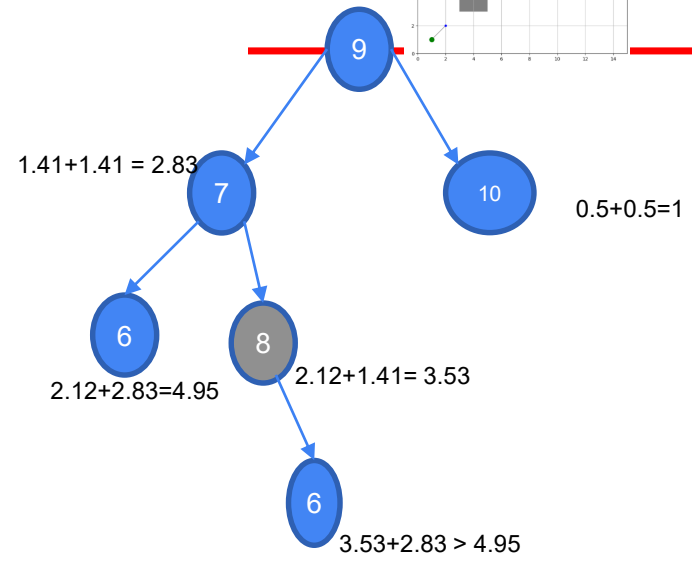
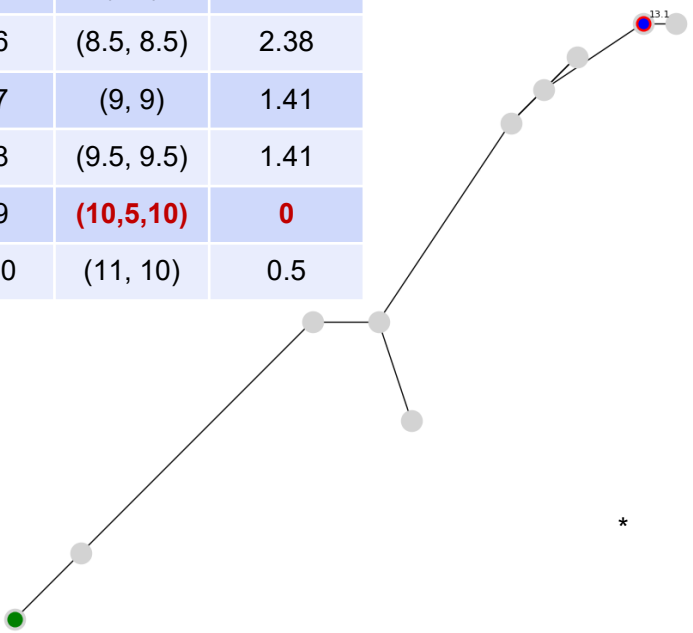
Start
Goal



D* Algorithm

Start = (1,1)
Goal = (10.5, 10)
 $K=f=g+h$
 $g(\text{Goal})=0$

n	State	h(n)
1	(1,1)	13.45
2	(2,2)	11.76
3	(5.5, 5.5)	7.78
4	(6.5, 6.5)	6.73
5	(7,4)	6.86
6	(8.5, 8.5)	2.38
7	(9, 9)	1.41
8	(9.5, 9.5)	1.41
9	(10.5,10)	0
10	(11, 10)	0.5



*

Expanding (10.5, 10) (k=13.1)

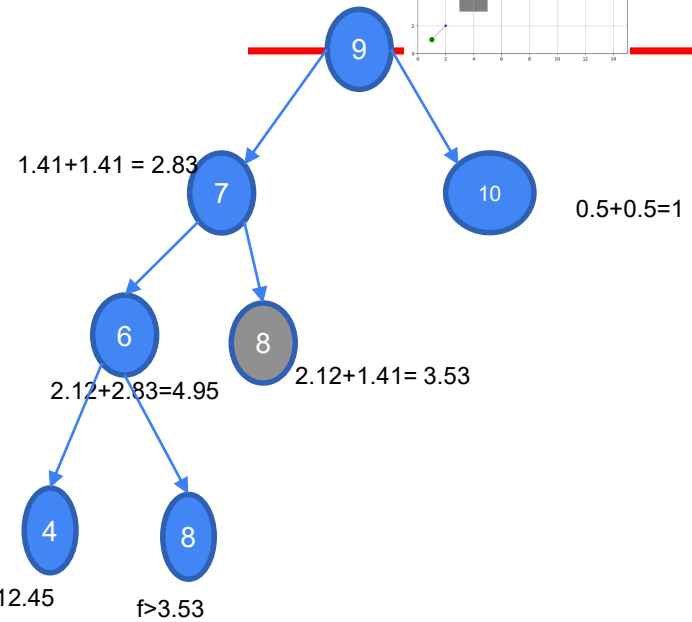
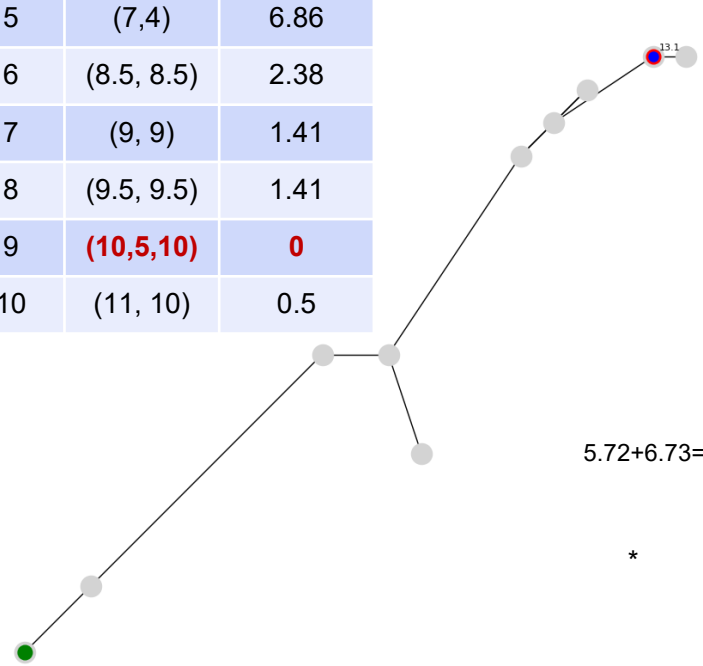
Start
Goal



D* Algorithm

Start = (1,1)
Goal = (10.5, 10)
 $K=f=g+h$
 $g(\text{Goal})=0$

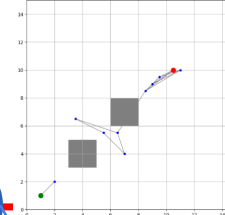
n	State	h(n)
1	(1,1)	13.45
2	(2,2)	11.76
3	(5.5, 5.5)	7.78
4	(6.5, 6.5)	6.73
5	(7,4)	6.86
6	(8.5, 8.5)	2.38
7	(9, 9)	1.41
8	(9.5, 9.5)	1.41
9	(10.5,10)	0
10	(11, 10)	0.5



*

Expanding (10.5, 10) (k=13.1)

Start
Goal

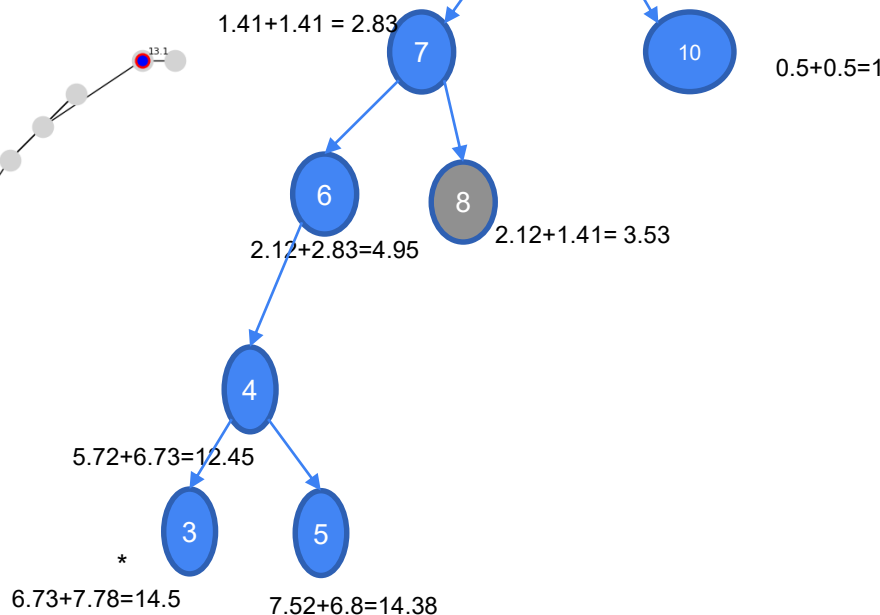


D* Algorithm

Start = (1,1)
Goal = (10.5, 10)

$K=f+g+h$
 $g(\text{Goal})=0$

n	State	h(n)
1	(1,1)	13.45
2	(2,2)	11.76
3	(5.5, 5.5)	7.78
4	(6.5, 6.5)	6.73
5	(7,4)	6.86
6	(8.5, 8.5)	2.38
7	(9, 9)	1.41
8	(9.5, 9.5)	1.41
9	(10.5,10)	0
10	(11, 10)	0.5



Expanding (10.5, 10) (k=13.1)

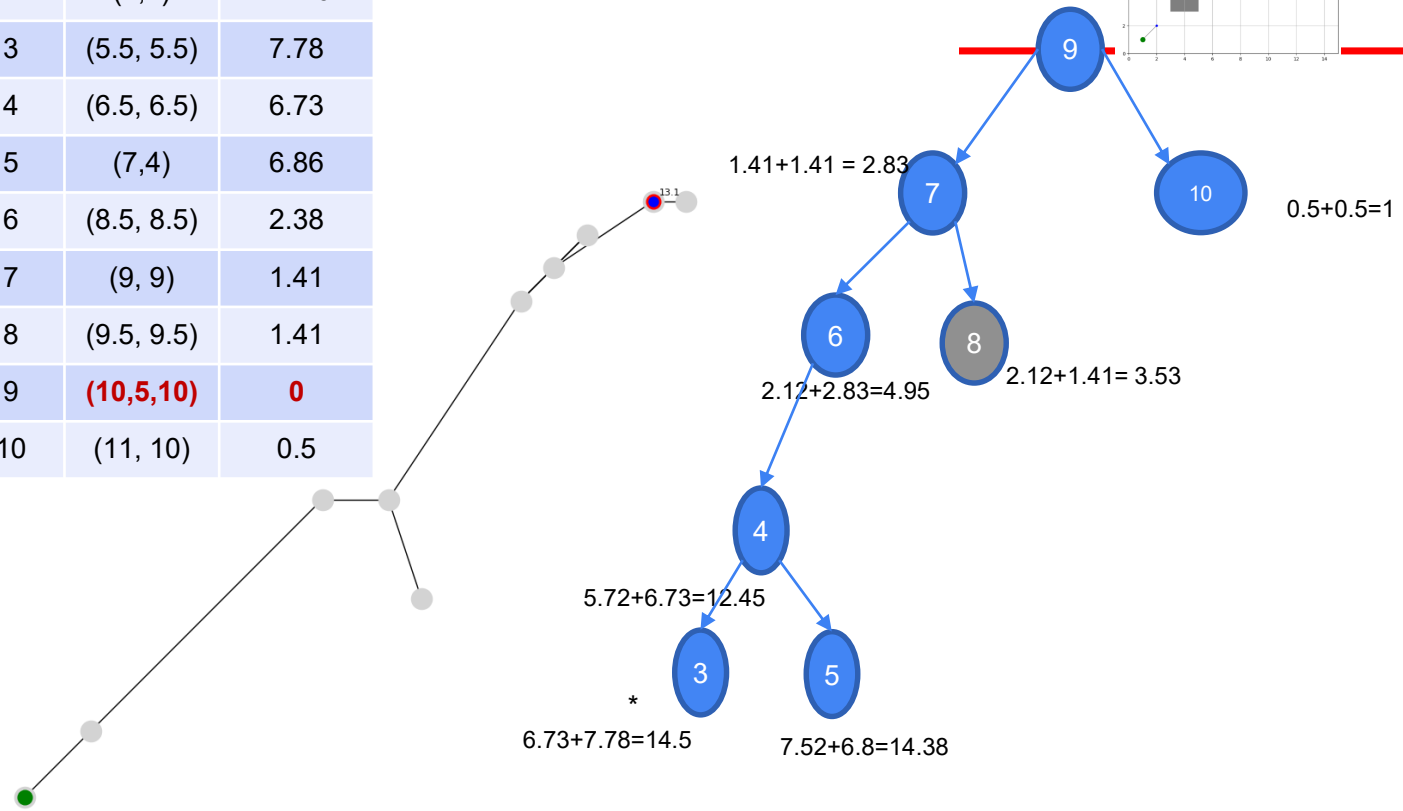
Start
Goal



D* Algorithm

Start = (1,1)
Goal = (10.5, 10)
 $K=f=g+h$
 $g(\text{Goal})=0$

n	State	h(n)
1	(1,1)	13.45
2	(2,2)	11.76
3	(5.5, 5.5)	7.78
4	(6.5, 6.5)	6.73
5	(7,4)	6.86
6	(8.5, 8.5)	2.38
7	(9, 9)	1.41
8	(9.5, 9.5)	1.41
9	(10.5,10)	0
10	(11, 10)	0.5



Expanding (10.5, 10) (k=13.1)

Start
Goal

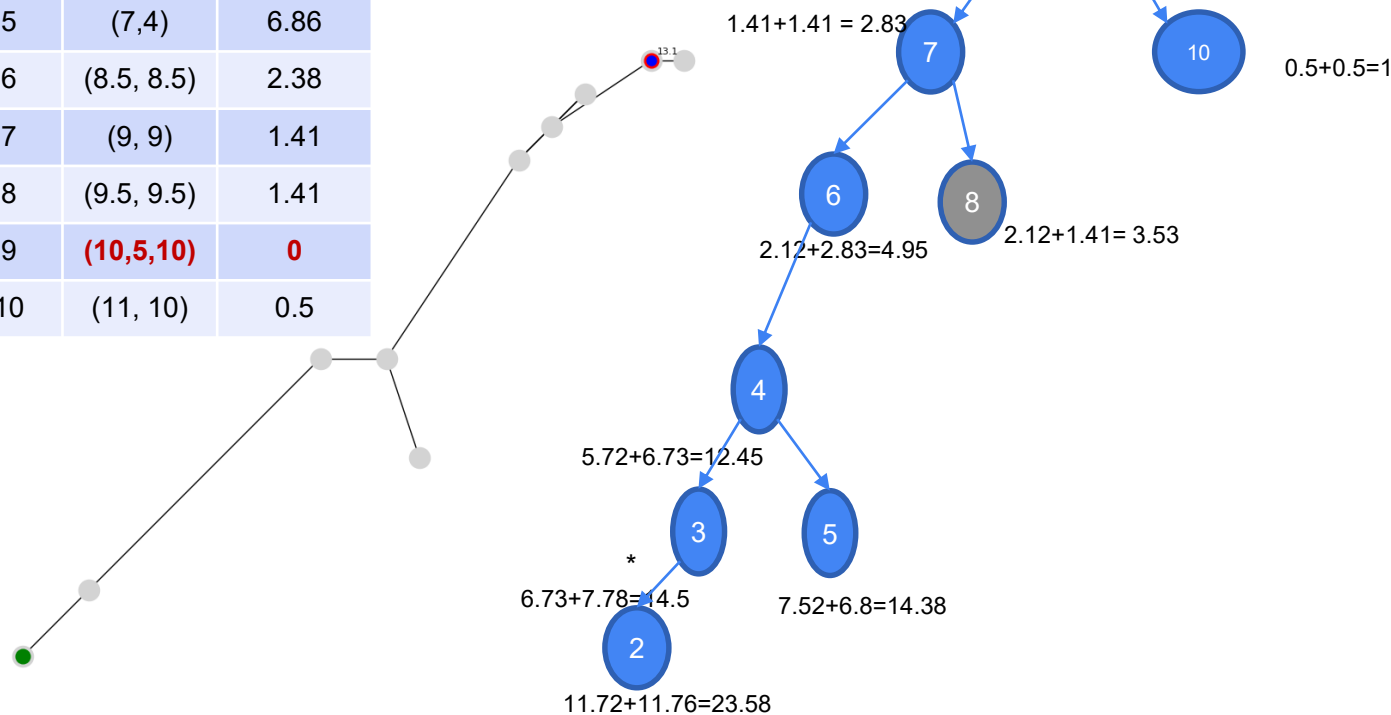


D* Algorithm

Start = (1,1)
Goal = (10.5, 10)

$K=f+g+h$
 $g(\text{Goal})=0$

n	State	h(n)
1	(1,1)	13.45
2	(2,2)	11.76
3	(5.5, 5.5)	7.78
4	(6.5, 6.5)	6.73
5	(7,4)	6.86
6	(8.5, 8.5)	2.38
7	(9, 9)	1.41
8	(9.5, 9.5)	1.41
9	(10.5, 10)	0
10	(11, 10)	0.5



Expanding (10.5, 10) (k=13.1)

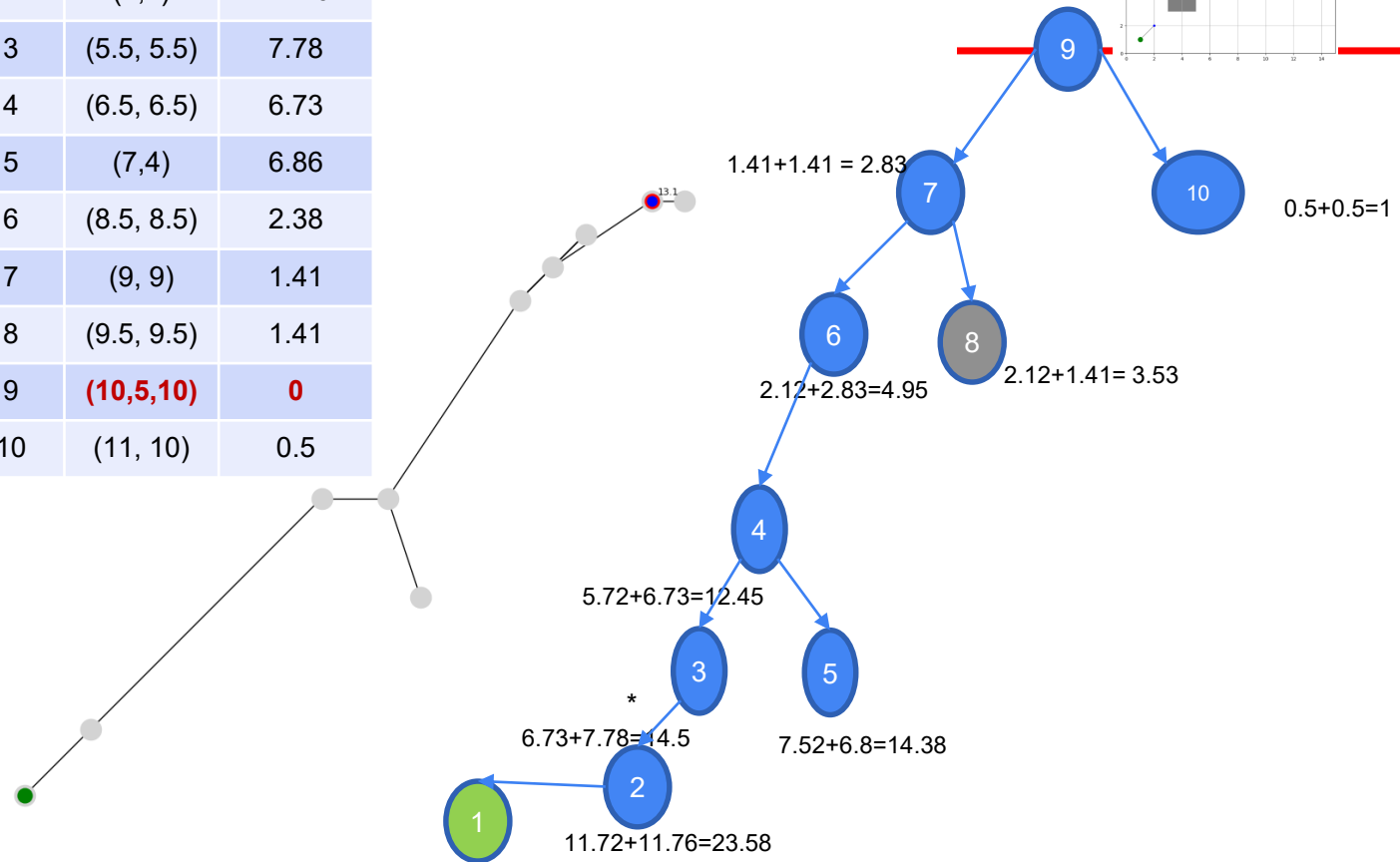
Start
Goal



D* Algorithm

Start = (1,1)
Goal = (10.5, 10)
 $K=f=g+h$
 $g(\text{Goal})=0$

n	State	h(n)
1	(1,1)	13.45
2	(2,2)	11.76
3	(5.5, 5.5)	7.78
4	(6.5, 6.5)	6.73
5	(7,4)	6.86
6	(8.5, 8.5)	2.38
7	(9, 9)	1.41
8	(9.5, 9.5)	1.41
9	(10.5, 10)	0
10	(11, 10)	0.5



Expanding (10.5, 10) (k=13.1)

Start
Goal



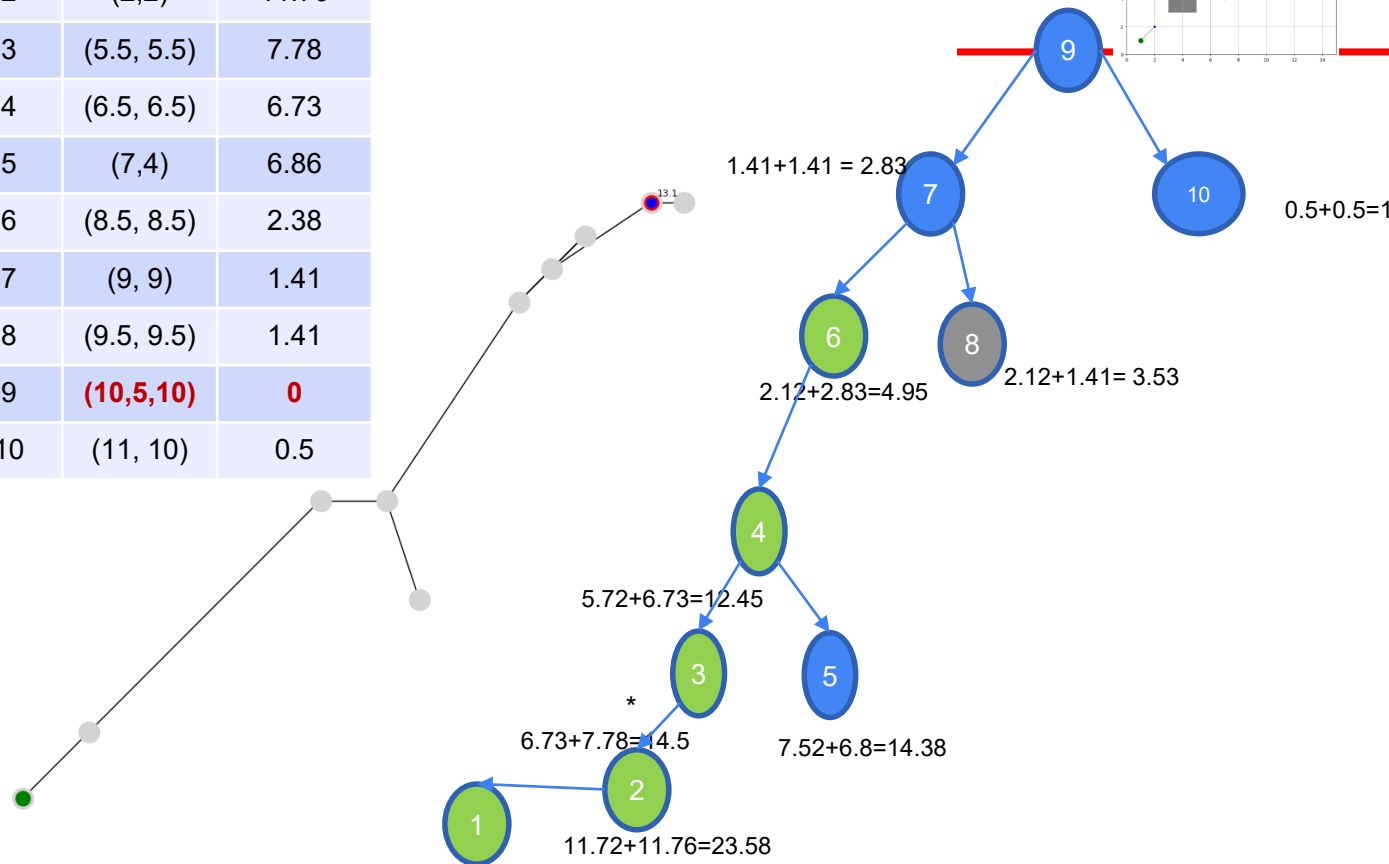
D* Algorithm

Start = (1,1)
Goal = (10.5, 10)

$K=f=g+h$
 $g(\text{Goal})=0$

Obstacle $S6 \leftrightarrow S7$

n	State	h(n)
1	(1,1)	13.45
2	(2,2)	11.76
3	(5.5, 5.5)	7.78
4	(6.5, 6.5)	6.73
5	(7,4)	6.86
6	(8.5, 8.5)	2.38
7	(9, 9)	1.41
8	(9.5, 9.5)	1.41
9	(10.5,10)	0
10	(11, 10)	0.5



Required Reading

- T2 Chapter 12

<https://erc-bpgc.github.io/handbook/>

Research Papers and Web Resources

Next Class Plan

- ✓ Swarm Intelligence
- ✓ Evolutionary Algorithms